

Design & Analysis of Algorithms

Practical Project Report

1.Introduction:

This project develops a smart-city transportation optimization system for Greater Cairo. The system combines graph algorithms, dynamic programming, and greedy methods to improve infrastructure design, route planning, emergency response, and public transport scheduling.

2.System Architecture:

2.1 Modules:

Data layer: stores neighborhoods, facilities, roads, traffic patterns, metro lines, bus routes, and transit demand.

Graph layer: builds a weighted undirected transportation graph.

Optimization layer: contains MST, shortest path, DP, and greedy algorithms.

Visualization layer: generates network diagrams for the demo.

Demo layer: Run sample scenarios and print optimization outputs.

2.2 Design Decisions:

- A weighted graph was used because roads and routes naturally map to vertices and weighted edges.
- Time-dependent edge weights model rush-hour variation.
- Separate algorithms were assigned to separate transportation tasks to keep the system modular and defensible in the report.

3. Algorithm Implementations

3.1 Modified Minimum Spanning Tree

- Base algorithm: Kruskal's algorithm.
- Modification: edge score includes distance, road condition, construction cost, capacity, population priority, and critical-facility preference.
- Goal: connect all strategic locations while reducing total weighted infrastructure cost.

3.2 Dijkstra's Algorithm

- Find the least-travel-time route between two locations.
- Edge weight = travel time estimated from distance, condition, and congestion.

3.3 A* Search for Emergency Routing

- Uses the same travel-time model as Dijkstra.
- Adds Euclidean heuristics to speed up routing toward medical targets.
- Give emergency vehicles a congestion discount to simulate sirens and intersection priorities.

3.4 Time-Dependent Shortest Path

- Uses morning, afternoon, evening, or night traffic data.
- Recompute path cost according to the selected traffic slot.

3.5 Dynamic Programming for Bus Allocation

- State: (route_index, remaining_extra_buses).
- Transition: assign 0..remaining buses to current route.
- Objective: maximize served daily passengers.

3.6 Dynamic Programming for Road Maintenance

- Modeled as 0/1 knapsack.
- Each repair candidate has a repair cost and a maintenance benefit.
- Objective: maximize total benefits under a fixed budget.

3.7 Greedy Traffic Signal Optimization

- Green time is assigned proportionally to queue length.
- A minimum green time constraint prevents starvation.

3.8 Emergency Vehicle Preemption

- Intersections are greedily ordered by congestion severity.
- Most congested intersections are cleared first.

4. Complexity Analysis

Component	Time Complexity	Space Complexity
Modified Kruskal MST	$O(E \log E)$	$O(V)$
Dijkstra	$O((V+E) \log V)$	$O(V)$
A*	$O((V+E) \log V)$ average with good heuristic	$O(V)$
Time-Dependent Dijkstra	$O((V+E) \log V)$	$O(V)$
Bus Allocation DP	$O(R * B^2)$ in this formulation	$O(R * B)$
Maintenance DP	$O(\text{Roads} * \text{Budget})$	$O(\text{Roads} * \text{Budget})$
Greedy Signal Optimization	$O(k)$	$O(k)$
Emergency Preemption Sorting	$O(k \log k)$	$O(k)$

5.Challenges Encountered

- The provided data mixes neighborhoods and facilities as nodes.
- Traffic data is time-varying but not fully real-time, so a slot-based model was used.
- Some facilities appear in facility data but not in road data, so the implementation adds a few short derived local access roads for those facilities to demonstrate emergency access realistically.
- Public transit data is aggregate, so DP uses a simplified capacity model suitable for course-level analysis.

6.Future Improvements

- Add live traffic ingestion and re-routing.
- Integrate metro transfers directly into multimodal routing.
- Add a GUI with selectable origin, destination, and time slot.
- Use multi-objective optimization for cost, fairness, and environmental impact.
- Extend emergency routing to multiple simultaneous incidents.

8.Conclusion

The project demonstrates how multiple algorithmic techniques can be integrated into one transportation optimization system. The final design is suitable because it is modular, explainable, data-driven, and easy to demonstrate.

Theoretical Report

Member 1: Ibrahim Mohamed Anwar

Modified Kruskal Algorithm for Infrastructure Network Design

Topic: Modified Minimum Spanning Tree for strategic road-network design

A graph $G = (V, E)$ models the city, where vertices are neighborhoods and facilities and edges represent roads. Classical MST finds a spanning tree minimizing total edge weight. Kruskal's algorithm sorts edges by non-decreasing weight and repeatedly adds the lightest edge that does not form a cycle. The cut property guarantees correctness: for any cut of the graph, the minimum-weight crossing edge belongs to some MST. Transportation-specific modifications include lower weights for existing roads, condition-adjusted costs, population-weighted endpoints, and priority for critical facilities such as airports, medical centers, and transit hubs.

Mathematical Foundations & Proof: Kruskal's is a greedy algorithm where correctness is guaranteed by the **cut property**: for any cut of the graph, the minimum-weight crossing edge belongs to some MST.

Transportation Modifications: Edge weights are adjusted based on existing infrastructure, population density at endpoints, and priority for hubs like airports or medical centers.

Performance & Optimization: Optimization is achieved through the **Union-Find** data structure with **path compression** to speed up cycle detection.

Complexity Analysis:

Metric	Complexity
Time	$O(E \log E)$
Space	$O(V)$

Comparison with Alternatives

- ☐ Prim's Algorithm: better suited for dense graphs and adjacency-matrix representations
- ☐ Steiner Tree: more realistic for strategic terminals but NP-hard and outside course scope
- ☐ Shortest-Path Tree: not suitable as it optimizes from a single source, not whole-network construction

Member 2: Moaz Mahmoud Gaber

Dijkstra's Algorithm for Standard Route Planning

Topic: Shortest-path routing with non-negative travel-time weights

For graph $G = (V, E)$ with non-negative weights $w(u, v)$, Dijkstra's algorithm computes the shortest path from source s to every other vertex. It maintains tentative distances and greedily finalizes the smallest one using a min-priority queue. Correctness follows from the fact that when a node u is extracted, its tentative distance equals the true shortest-path distance—any alternative path must be at least as large due to non-negative weights. Edge weights represent travel time derived from distance, road condition, capacity, and congestion levels.

Mathematical Foundations & Proof: Uses the **relaxation operation**. Correctness relies on non-negative weights: when a node is finalized, its tentative distance must be the true shortest path because any alternative path would only add positive weight.

Transportation Modifications: Edge weights represent travel time influenced by road conditions, capacity, and current congestion levels.

Performance & Optimization: Performance is optimized through efficient priority queue extraction and road closure modeling via edge removal.

Complexity Analysis:

Metric	Complexity
Time	$O((V + E) \log V)$
Space	$O(V)$

Comparison with Alternatives:

- ❑ Bellman-Ford: handles negative weights but is significantly slower — $O(VE)$
- ❑ Floyd-Warshall: solves all-pairs but is too expensive for larger graphs — $O(V^3)$
- ❑ A* Search: faster for goal-directed queries when a good heuristic exists.

Member 3: Omar Tamer Abdullah

A* Search for Emergency Vehicle Routing

Topic: Heuristic shortest-path search for emergency response optimization

A* evaluates nodes with the combined cost function $f(n) = g(n) + h(n)$, where $g(n)$ is the actual cost from the source to node n and $h(n)$ estimates the remaining cost to the target. If the heuristic h is admissible (never overestimates) and consistent (satisfies the triangle inequality), A* returns the optimal path. Euclidean distance is used as the heuristic, which is always admissible in a road network. Emergency vehicles receive lower effective congestion penalties, enabling faster clearing of routes to hospitals and emergency-sensitive facilities.

Mathematical Foundations & Proof: Evaluates nodes using $f(n) = g(n) + h(n)$. It is guaranteed to find the optimal path if the heuristic $h(n)$ is **admissible** (never overestimates) and **consistent**.

Transportation Modifications: Uses **Euclidean distance** as the heuristic for road networks. Emergency vehicles are assigned lower effective congestion penalties to prioritize their routes.

Performance & Optimization: Optimization focuses on reducing the "search space" by using a heuristic that reflects the spatial reality of the city.

Complexity Analysis:

Metric	Complexity
Worst Case Time	$O((V + E) \log V)$
Practical Time	Often fewer states explored
Space	$O(V)$

Comparison with Alternatives:

- ☐ Dijkstra: no heuristic, expands more nodes, equivalent to A* with $h(n) = 0$
- ☐ Greedy Best-First Search: faster in practice but not guaranteed to find the optimal path

- ❑ Bidirectional Search: useful for fixed source-target pairs but more complex to implement cleanly

Member 4: Ahmed Ibrahim Mohamed

Time-Dependent Shortest Path Algorithm

Topic: Routing with time-varying edge weights to model congestion patterns

Instead of a static weight function $w(e)$, the time-dependent model uses $w(e, t)$ where the cost of traversing an edge depends on the departure time slot. This accurately models real-world congestion patterns throughout the day. If the network satisfies the FIFO property—leaving earlier cannot make you arrive later on the same edge—generalized Dijkstra remains valid. The project uses four time slots (morning, afternoon, evening, night) to approximate congestion, with effective speed dropping proportionally as the congestion ratio (flow / capacity) increases.

Mathematical Foundations & Proof: Models weights as a function of time. Correctness is maintained if the network satisfies the **FIFO property** (leaving earlier ensures arriving no later than someone leaving after you).

Transportation Modifications: Implements **four time slots** (morning, afternoon, evening, night) where speed drops as the congestion ratio (flow/capacity) increases.

Performance & Optimization: Uses a discrete slot model to balance realism with computational speed.

Complexity Analysis:

Metric	Complexity
Time (slot model)	$O((V + E) \log V)$
Space	$O(V)$

Comparison with Alternatives:

- ❑ Static Dijkstra: simpler but ignores rush hour and time-of-day effects
- ❑ Time-Expanded Networks: more accurate but produce a much larger graph with higher memory cost

- ❑ Simulation-Based Routing: flexible and realistic but computationally much heavier

Member 5: Mohamed Ahmed Abdeltawab

Dynamic Programming for Public Transportation Scheduling

Topic: Optimal allocation of extra buses under limited fleet resources

Dynamic programming solves optimization problems exhibiting overlapping subproblems and optimal substructure. The state $DP(i, b)$ represents the maximum passengers served from route i onward with b extra buses remaining. The recurrence is: $DP(i, b) = \max_{x \in [0, b]} \text{served}(i, x) + DP(i+1, b-x)$. Correctness follows from the principle that the best global allocation decomposes into the best choice for the current route plus the best allocation of remaining buses. Passenger demand from the dataset determines route importance, and the objective is to maximize covered daily passengers.

Mathematical Foundations & Proof: Relies on **optimal substructure** and **overlapping subproblems**. The recurrence $DP(i, b) = \max \{\text{served}(i, x) + DP(i+1, b-x)\}$ ensures global optimality through local sub-decisions.

Transportation Modifications: Fleet allocation is driven by passenger demand data to maximize total covered daily passengers.

Performance & Optimization: Optimization involves DP table construction and traceback to find the exact allocation efficiently.

Complexity Analysis:

Metric	Complexity
Time	$O(R \times B^2)$
Space	$O(R \times B)$

Variable Definitions:

- ❑ R = number of bus routes in the network
- ❑ B = total number of extra buses available for allocation

Comparison with Alternatives:

- ❑ Greedy Allocation by Highest Demand: simpler but may miss optimal combinations across routes

- ❑ Linear Programming: theoretically more powerful but outside the course scope
- ❑ Brute Force: exact but exponential in complexity — $O(B^R)$

Member 6: Mohamed Islam

Greedy Traffic Signal Optimization

Topic: Greedy-based intersection signal timing for real-time traffic control

Greedy algorithms build a solution incrementally by making the best immediate local choice. For signal timing, green time is assigned proportionally to the queue size at each lane group. This approach is attractive for real-time traffic control because decisions are simple, fast, and interpretable. Unlike MST or Dijkstra, greedy signal timing does not always guarantee a global optimum—however, it performs well in practice for local intersection management where the cost of computation must be near-instantaneous.

Mathematical Foundations & Proof: Based on the **greedy-choice principle**, making locally optimal decisions at each step. It does not always guarantee a global optimum but is effective for real-time control.

Transportation Modifications: Green time is assigned proportionally to current **lane queue sizes**.

Performance & Optimization: Incorporates a **minimum green-time constraint** to prevent lane starvation while maintaining near-instantaneous decision-making.

Complexity Analysis:

Metric	Complexity
Signal Allocation Time	$O(k)$
Space	$O(k)$

Variable Definitions:

- ❑ k = number of lane groups or intersections under consideration

Comparison with Alternatives:

- ❑ DP / Optimization-Based Control: more globally optimal but too slow for real-time use

- ❑ Reinforcement Learning: adaptive and powerful but hard to justify theoretically in this course
- ❑ Fixed-Time Control: simplest to implement but completely unresponsive to actual traffic conditions

Member 7: Yassin Nader Ahmed Ibrahim

Emergency Vehicle Preemption System

Topic: Priority-based emergency intersection control for fast corridor clearing

Emergency vehicle preemption temporarily overrides normal signal timing to grant unobstructed passage through intersections along an emergency route. The system orders intersections by a congestion-priority score, then clears them in that order using a greedy sorting strategy. This enables the fastest possible corridor clearing when seconds matter. Congestion severity acts as the priority score, ensuring that the most blocked intersections are addressed first. The system is designed for real-time responsiveness and integrates directly with the A* emergency routing module.

Mathematical Foundations & Proof: Uses a **greedy sorting strategy** to clear corridors. It identifies the most congested intersections along a route and prioritizes them for override logic.

Transportation Modifications: Uses a **congestion-priority score** to address the most "blocked" intersections first.

Performance & Optimization: Directly integrates with *Member 3's A routing** to ensure the preemption logic follows the active emergency path in real time.

Complexity Analysis:

Metric	Complexity
Sorting Intersections Time	$O(k \log k)$
Space	$O(k)$

Variable Definitions:

- ❑ k = number of intersections along the emergency vehicle corridor

Comparison with Alternatives:

- ☐ Fixed-Time Systems: no responsiveness to emergency vehicles, delays are unpredictable
- ☐ Adaptive Signal Control: reacts to general congestion but not tuned for emergency priority
- ☐ Manual Override: operator-controlled but not scalable to an entire route in real time

Team Members:

- **Mohamed Islam 22101341**
- **Yassin Nader Ahmed 23101504**
- **Moaz Mahmoud Gaber 23101486**
- **Omar Tamer Abdullah 23101428**
- **Ibrahim Mohamed Anwar 23101346**
- **Mohamed Ahmed Abdeltawab 22101437**
- **Ahmed Ibrahim Mohamed 21100777**